

Pre-Screening Assignment

Frontend React Internship • Complete within 24 hours

■ Time to complete	4 – 8 hours
■ What to submit	GitHub link + 5-min screen recording
■ Deadline	Within 24 hours of receiving this
■ Difficulty	Basic to intermediate React

What You Are Building

Build a **Photo Gallery Web App** in React. The app fetches photos from a free public API, displays them in a grid, lets the user search by keyword, and allows marking photos as favourites. That is the complete scope — do not add anything beyond what is listed below.

Requirements

1 Project Setup

Create a React + Vite project. Use Tailwind CSS for all styling. No other UI libraries (no MUI, no Ant Design, no Bootstrap).

Why we ask this: We use React + Vite + Tailwind in our codebase. This checks if you can set it up from scratch.

2 Fetch Photos from API

Use the Picsum Photos API (<https://picsum.photos/v2/list?limit=30>) to fetch 30 photos on load. Show a loading spinner while fetching. Show an error message if the fetch fails.

Why we ask this: We fetch data from remote APIs throughout our app. Loading and error states are not optional.

3 Display in a Responsive Grid

Show all photos in a grid. On desktop: 4 columns. On tablet: 2 columns. On mobile: 1 column. Each card shows the photo, the author name, and a heart icon button.

Why we ask this: Our gallery renders image grids across all screen sizes. Responsiveness is a basic requirement.

4 Search Filter

Add a search input at the top. As the user types, filter the displayed photos by author name in real-time. No page reload. No API call on search — filter the already-fetched data.

Why we ask this: Real-time filtering of fetched data is used in our Dashboard's search bar.

5 Favourites with useReducer

The heart icon toggles a photo as a favourite. Manage the favourites list using useReducer — not useState. Favourited photos must persist across page refreshes using localStorage.

Why we ask this: We use reducers for complex state. localStorage persistence is core to how we save user data.

6 Custom Hook

Extract the fetch logic into a custom hook called useFetchPhotos. It must return: photos, loading, error. The Gallery component must use this hook — not fetch data directly.

Why we ask this: We use custom hooks throughout our codebase to separate data logic from UI.

7 useCallback + useMemo

Use useCallback on the search filter handler. Use useMemo to compute the filtered photo list. Be ready to explain in your video why you used each one.

Why we ask this: Performance hooks are used extensively in our gallery. Understanding them is mandatory.

Screen Recording — 5 Minutes

Record yourself explaining your code. This is mandatory — submissions without a recording will not be reviewed.

1 Show the app working

Open the app in the browser. Show photos loading, search filter working, and favourites toggling and persisting after refresh.

2 Open your useFetchPhotos hook

Read out the hook and explain: what does it return, and what happens if the API fails?

3 Open your useReducer code

Explain: what actions does your reducer handle? Why did you choose useReducer instead of useState here?

4 Open the component where you used useCallback and useMemo

Explain in plain words: what problem does each one solve in your code? What would break if you removed them?

5 Show one thing that was difficult

Pick one part that took you time to figure out. Explain what the problem was and how you solved it.

Free recording tools: Windows — press Win + G. Mac — open QuickTime → New Screen Recording. Any OS — OBS Studio (free, no time limit). Upload to Google Drive or YouTube (unlisted) and paste the link.

Important Rules

- **Do NOT use any component libraries.**
Tailwind only. No MUI, no Bootstrap, no Ant Design.
- **Do NOT use class components.**
Functional components with hooks only.
- **Do NOT skip useReducer.**
Favourites state must use useReducer. useState is not accepted here.

- **Do NOT submit without a recording.**
Code with no video will not be reviewed at all.
- **Do NOT add extra features.**
Stick to the 7 requirements. Extra features do not improve your score.
- **AI tools are allowed.**
But you must be able to explain every line in your video. If you cannot explain it, do not include it.

How to Submit

- 1 Push your code to a public GitHub repository.
- 2 Upload your screen recording to Google Drive or YouTube (set to unlisted).
- 3 Fill the Google Form (link shared separately) with: your GitHub link, your recording link, and one line describing the hardest part of the assignment.
- 4 Submit within 24 hours of receiving this document.

What we are looking for: We are not looking for a perfect app. We are looking for someone who understands what they built and can explain it clearly. A simple, working app with a confident video explanation beats a complex app where you cannot explain a single line.